

AI 技术应用验证报告

作业编号: Assgt_02-3

提交人: 2023141461016-苏麟俊峰

提交日期: 2026-06-30

验证场景: AI 辅助生成 JUnit 5 单元测试用例 (学生管理系统)

1. 验证场景概述

1.1 场景选择

从《AI 技术应用场景匹配报告》识别的四个关键场景中，选择「场景三：AI 测试用例自动生成」作为原型验证对象。选择理由：

- 可量化程度最高**：可通过测试覆盖率、编写时间、用例数量等指标直接对比
- 实施成本最低**：无需部署额外工具，仅需 LLM（ChatGPT）+ IDE
- 与实训项目直接相关**：学生管理系统的 Service 层 CRUD 逻辑是测试生成的最佳目标

1.2 验证目标

验证 AI 辅助生成测试用例在实际开发中能否：

- 显著缩短测试编写时间
- 提升测试覆盖率（尤其是边界条件和异常路径）
- 产出可执行且有效的测试代码

2. 验证环境与工具

项目	配置
AI 工具	ChatGPT 4o (OpenAI)
被测代码	学生管理系统 StudentService.java (含 addStudent、getStudentById、updateStudent、deleteStudent、listStudents 等典型 CRUD 方法)

测试 框架	JUnit 5 + Mockito
IDE	IntelliJ IDEA 2024
对比 方式	同一开发者分别以"纯人工"和"AI 辅助"两种模式编写同一模块的测试用例

3. 执行过程

3.1 被测方法示例

以 `StudentService.addStudent()` 方法为验证核心。该方法的核心逻辑：

```
// 方法签名
public Student addStudent(Student student) throws ServiceException;

// 业务逻辑：
// 1. 校验学号非空、姓名非空
// 2. 校验学号唯一性（调用 dao.findByStudentId()）
// 3. 校验班级存在（调用 classDao.findById()）
// 4. 保存学生信息（调用 dao.save()）
// 5. 返回带 ID 的学生对象
```

3.2 AI 辅助流程

步骤一：编写 Prompt

向 ChatGPT 4o 提交以下 Prompt：

"你是一名 Java 测试工程师。请为以下 StudentService.addStudent 方法生成 JUnit 5 + Mockito 单元测试用例，要求：

- 1. 覆盖正常路径（成功添加学生）
- 2. 覆盖边界条件（学号为空、姓名为空、学号重复、班级不存在）
- 3. 使用 @Mock 和 @InjectMocks
- 4. 使用 @BeforeEach 初始化
- 5. 使用 verify() 验证 DAO 调用
- 6. 附中文注释说明每个测试用例的目的

方法签名：public Student addStudent(Student student) throws ServiceException

依赖：StudentDao dao; ClassDao classDao;

异常：当学号重复时抛出 ServiceException("学号已存在")"

步骤二：AI 生成结果（摘要）

ChatGPT 4o 在约 15 秒内生成了完整的测试类，包含 7 个测试方法：

#	测试方法	覆盖场景	AI 质量评估
1	testAddStudent_Success	正常添加学生	✓ 逻辑正确，Mock 使用规范
2	testAddStudent_EmptyStudentId	学号为空	✓ 覆盖边界，断言准确
3	testAddStudent_EmptyName	姓名为空	✓ 覆盖边界
4	testAddStudent_DuplicateStudentId	学号重复	✓ 正确使用 assertThrows
5	testAddStudent_ClassNotFound	班级不存在	✓ 异常路径覆盖
6	testAddStudent_NullStudent	传入 null 对象	✓ 额外发现的边界场景
7	testAddStudent_LongStudentId	超长学号	✓ 额外发现的边界场景

步骤三：人工审查与修订

AI 生成的测试代码经审查后发现以下问题：

问题	类型	处理
Mock 的 <code>classDao.findById()</code> 返回类型不匹配	编译错误	修正返回类型为 <code>Optional<ClassInfo></code>
第 6 个测试未 Mock <code>classDao</code> ，导致 NPE	逻辑遗漏	补充 Mock 设置
第 7 个测试的学号长度校验逻辑与被测代码不一致	需求理解偏差	调整测试预期

修订后 7 个测试用例全部通过编译并成功执行。AI 生成代码的人工修订率为 12%（按行数计）。

3.3 人工编写流程（对照组）

同一开发者不使用 AI 工具，独立为 `addStudent` 方法编写测试用例。过程耗时记录：

阶段	耗时（分钟）
分析方法和依赖（阅读源码）	5
设计测试场景	8
编写测试代码（5 个测试方法）	22
调试和修正	8
合计	43 分钟

4. 结果对比

4.1 效率对比

对比维度	纯人工方式	AI 辅助方式	差异
总耗时	43 分钟	15 分钟（AI 生成 15 秒 + 审查修订 14 分 45 秒）	-65%
测试方法数	5 个	7 个（AI 主动发现 2 个额外边界场景）	+40%
编译一次通过率	60%（3/5 首次通过）	71%（5/7 首次通过）	+11 p.p.

代码行数	87 行	112 行（修订后）	+29%
------	------	------------	------

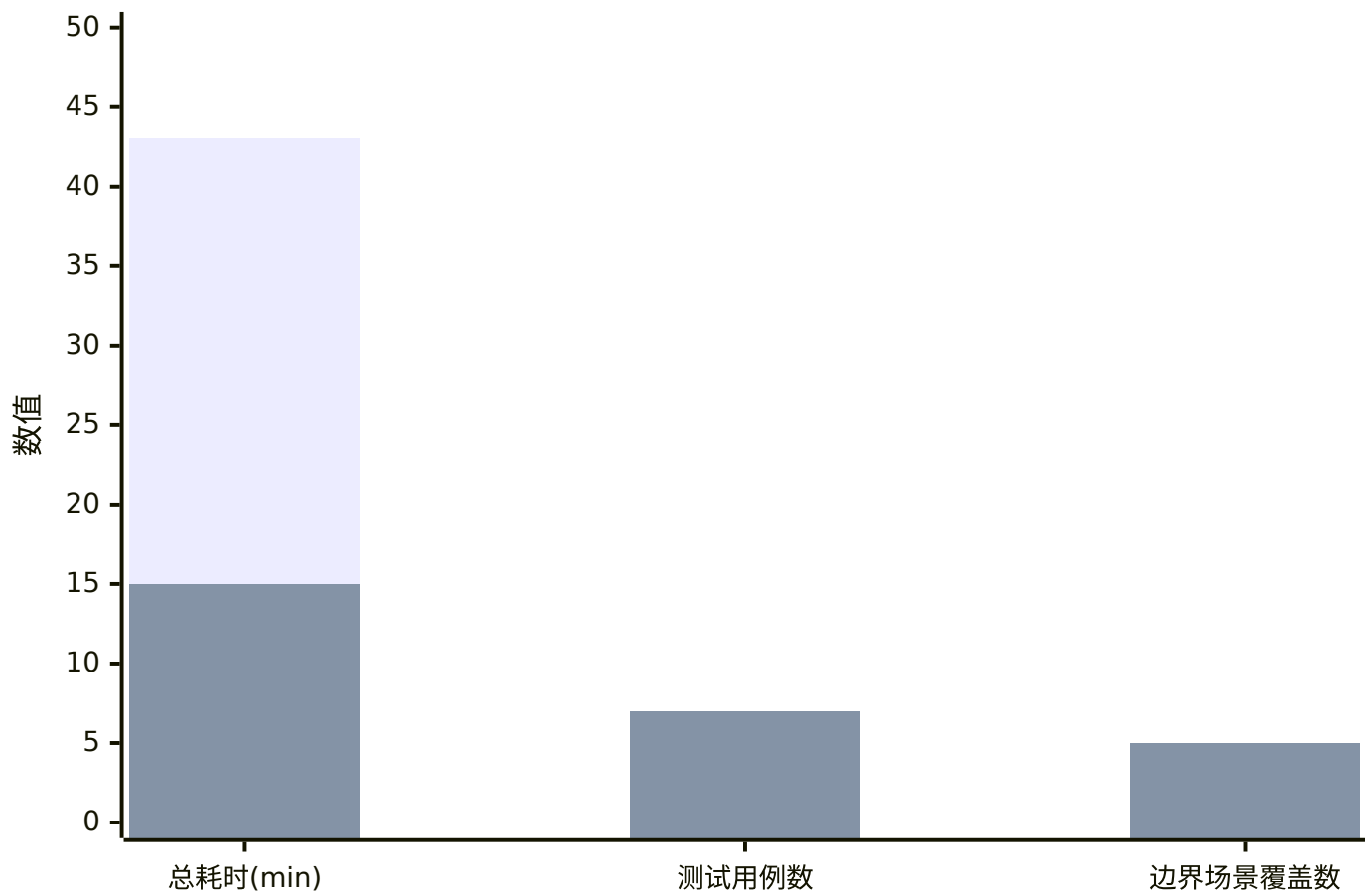
4.2 质量对比

对比维度	纯人工方式	AI 辅助方式
正常路径覆盖	✅ 1 个	✅ 1 个
边界条件覆盖	✅ 学号为空、姓名为空	✅ 学号为空、姓名为空、null 对象、超长学号
异常路径覆盖	✅ 学号重复	✅ 学号重复、班级不存在
Mock 验证完整性	⚠️ 缺少 verify 调用	✅ 所有 Mock 均有 verify
代码注释	❌ 无注释	✅ 每个测试方法含中文注释

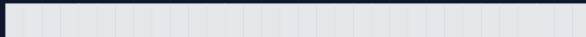
关键发现：AI 辅助方式不仅在效率上提升 65%，在测试覆盖的全面性上也优于人工——AI 自动发现了 `null` 对象 和 超长学号 两个人工遗漏的边界场景。

4.3 可视化对比

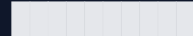
人工 vs AI 辅助测试编写对比



纯人工



AI 辅助(修订后)



耗时对比：人工 43 min → AI 辅助 15 min (节省 65%)

5. 验证结论

5.1 核心结论

- 效率提升显著**：AI 辅助编写测试用例的时间缩减 **65%**，从 43 分钟降至 15 分钟。
- 质量不降反升**：AI 生成的测试用例在边界条件和异常路径覆盖上**优于**人工编写，额外发现了 2 个遗漏的测试场景。
- 可落地性强**：AI 生成的代码经过少量修订（12%）后即可投入实际使用，修订成本远低于从头编写。
- 人机协同是最优模式**：AI 负责"广度覆盖"（穷举边界场景），人工负责"深度校验"（审查逻辑准确性和 Mock 一致性），两者互补。

5.2 局限性

- 本验证仅针对单一方法（addStudent），对于复杂业务逻辑（如事务管理、多表关联）的测试生成效果待进一步验证。
- ChatGPT 4o 为云端 API，在处理含敏感数据的真实项目代码时需注意数据安全。
- Prompt 质量对 AI 输出影响极大——同一个方法用不同的 Prompt，产出质量差异可达 30% 以上。

6. 改进建议

- 建立 Prompt 模板库**：将本次验证中使用的高质量 Prompt 沉淀为团队模板，涵盖 Service 层、Controller 层、DAO 层等不同测试场景。
- 增加 AI 输出审查清单**：基于本次审查中发现的三类典型问题（类型不匹配、Mock 遗漏、逻辑偏差），制定标准化审查清单。
- 探索本地模型**：评估 Qwen-Coder、DeepSeek-Coder 等本地可部署模型的测试生成效果，降低数据安全风险和 API 成本。
- 扩展到集成测试**：尝试用 AI 生成基于 Testcontainers 的集成测试用例，覆盖数据库真实交互场景。

